

Chapter 2. Flow Control

Develop code that implements an if or switch statement; and identify legal argument types for these statements.

The syntax of the switch statement is extended ever-so-slightly. The type of the Expression is now permitted to be an enum class. (Note that `java.util.Enum` is not an enum class.) A new production is added for SwitchLabel:

```
SwitchLabel:  
    case EnumConst :  
  
EnumConst:  
    Identifier
```

The Identifier must correspond to one of UNQUALIFIED enumeration constants.

Here is a slightly more complex enum declaration for an enum type with an explicit instance field and an accessor for this field. Each member has a different value in the field, and the values are passed in via a constructor. In this example, the field represents the value, in cents, of an American coin.

```
public enum Coin {  
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);  
  
    Coin(int value) {  
        this.value = value;  
    }  
    private final int value;  
    public int getValue() { return value; }  
}
```

Switch statements are useful for simulating the addition of a method to an enum type from outside the type. This example "adds" a color method to the Coin class, and prints a table of coins, their values, and their colors.

```
import static java.lang.System.out;  
  
public class CoinTest {  
    public static void main(String[] args) {  
        for (Coin c : Coin.values()) {  
            out.println(c + ":  \t" + c.getValue() + "c  \t" + color(c));  
        }  
  
        private enum CoinColor {  
            COPPER, NICKEL, SILVER  
        }  
  
        private static CoinColor color(Coin c) {  
            if (c == null) {  
                throw new NullPointerException();  
            }  
  
            switch (c) {  
                case Coin.PENNY: {} // Compile error! Must be UNQUALIFIED !!!  
                case PENNY:
```

```

        return CoinColor.COPPER;
    case NICKEL:
        return CoinColor.NICKEL;
    case DIME:
        return CoinColor.SILVER;
    case QUARTER:
        return CoinColor.SILVER;
    case 2: {}          // Compile error !!!
                      // Type mismatch: cannot convert from int to Coin
    }

    throw new AssertionError("Unknown coin: " + c);
}
}

```

Running the program prints:

```

PENNY:      1c      COPPER
NICKEL:     5c      NICKEL
DIME:   10c      SILVER
QUARTER:   25c      SILVER

```

[Prev](#)

Given a set of classes and superclasses, develop constructors for one or more of the classes. Given a class declaration, determine if a default constructor will be created, and if so, determine the behavior of that constructor. Given a nested or non-nested class listing, write code to instantiate the class.

[Up](#)

[Home](#)

[Next](#)

Develop code that implements all forms of loops and iterators, including the use of for, the enhanced for loop (for-each), do, while, labels, break, and continue; and explain the values taken by loop counter variables during and after loop execution.



SCDJWS 1.4
Quiz



ICAD
Study Guide



Magnet
Mocker

SCWCD 1.4
Study Guide

SCBCD 1.3
Study Guide

SCDJWS 1.4
Study Guide

IBM Test 287
Study Guide